

Real-world (score an answer that isn't exact) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper implementation), **Advanced** (how you'd actually trust a judge before relying on it). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You can't check an LLM's answer with `==` because it could say the same correct thing in a hundred different ways. So instead, write down what a *correct* answer must actually contain or do — a short checklist — before you look at any real output.

Then there are two ways to check that checklist: write plain Python code that checks for specific words or patterns, or ask another LLM call to read the answer and the checklist and decide pass/fail. That second one is called "LLM-as-a-judge."

Start by picking one real task from Project 4 and writing your checklist for it in plain English, first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The order matters here more than in any other exercise in this document: **write your pass/fail rules before you look at a single real output**. If you write the rules after seeing the answer, you'll unconsciously write rules that just describe whatever the model happened to say — which tells you nothing about whether it's actually right.

Two ways to check rules against text that varies: - **Property checks in plain Python** — cheap, fast, no extra API call, but only work for rules you can express in code: "contains the word X," "is valid JSON," "is under N words." - **LLM-as-a-judge** — a second model call, given the original task, the answer, and your rules, asked to score or pass/fail it. Needed for rules that require judgment: "is this actually helpful," "does this correctly summarize the source," "is the tone appropriate."

Write 2-3 concrete rules for one real Project 4 task now, and mark each one as "checkable in plain code" or "needs a judge."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the uncomfortable fact from this document's Core Concepts, worth sitting with before you write a single line of judge code: **the judge is not a neutral referee. It's a piece of software you built, with a prompt, and it can be wrong exactly like anything else you build.** A badly written judge that rewards long, confident-sounding answers no matter how correct they are will happily approve bad answers — and because its output *looks* like a verdict, it's easy to trust it more than it deserves.

So the real design question isn't just "how do I build a judge" — it's **"how do I know my judge is actually any good before I let it gate anything real?"** The way you'd answer that for a human reviewer (give them a few cases where you already know the right answer, and see if they agree) is exactly the way you check a judge too.

The extra pieces needed:

- **A small calibration set** — 4-6 answers where *you* already know the correct verdict: 2-3 obviously good answers, and 2-3 obviously bad ones (wrong fact, way too long, ignores the question). Run your judge against these before trusting it on anything real.
- **Self-consistency check** — call the judge on the *same* task and answer 3 times. If it says PASS twice and FAIL once on identical input, your judge prompt is too vague or your model's temperature is too high for a scoring task — a judge that isn't consistent with itself can't be consistent about anything else either.
- **Position/length bias** — a known failure mode of LLM judges is favoring the longer or more confident-sounding of two answers, independent of correctness. If your judge ever compares two answers side by side, try swapping their order and confirm the verdict doesn't flip — that's evidence of order bias, not quality signal.
- **Low temperature** (like 0 or close to it) **for the judge's own model call** — a judge that varies its verdict due to sampling randomness, not due to a genuine difference in the answer, is adding noise you don't want in a system whose whole job is producing a trustworthy score.

Before writing `score_output()`, sketch 2 obviously-bad answers you'd expect your judge to catch — you'll use them to sanity-check the judge once it's built.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both focus on building the judge correctly — write rules first, choose property checks vs. a judge call per rule. Advanced assumes you've built it, and asks the harder question underneath: how do you know it's trustworthy? A calibration set, a self-consistency check, and controlling for known bias patterns (position, length) are what separate "I wrote a judge" from "I have evidence my judge actually works," which matters a great deal once this judge is gating whether a prompt change ships.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
rules = ["mentions the correct category", "under 100 words", "no made-up facts"]

function judge(task, answer, rules):
    build a prompt: here is the task, here is the answer, here are the rules,
        reply PASS or FAIL and one reason
    call the model with that prompt
    read back PASS/FAIL and the reason
    return them

test:
    answer = run the real task through Project 4
    result = judge(task, answer, rules)
    assert result is PASS
```

Here's almost the whole thing — just try running it and reading it line by line:

```
def build_judge_prompt(task, answer, rules):
    rules_text = "\n".join("- " + r for r in rules)
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text + "\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining"
    )
```

Expected output if you run just this: nothing — defining a function doesn't run it. Now write the function that calls the model with this prompt and parses `PASS / FAIL` back out.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
rules = [
    "mentions the correct category",
    "is under 100 words",
    "does not state a fact that isn't in the source material",
]

function score_output(task, answer, rules) -> JudgeResult:
    judge_prompt = build a prompt containing:
        the original task
        the answer being scored
        the rules, as a numbered list
        instructions: reply with PASS or FAIL, then one sentence explaining why
    response = call the model with judge_prompt
    parse response into (verdict, reason)
    return JudgeResult(verdict, reason)
```

```
def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = "\n".join(f"- {r}" for r in rules)
    return (
        f"Task: {task}\n"
        f"Answer to check: {answer}\n"
        f"Rules the answer must follow:\n{rules_text}\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining"
    )
```

What's missing: `score_output()`, which calls the model with this prompt and parses the reply into a `verdict` and a `reason`. Write it yourself — think about what happens if the model's reply doesn't start with exactly `PASS` or `FAIL` — before moving to Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

build_judge_prompt(task, answer, rules) – same as Intermediate, called with tempera

score_output(task, answer, rules) -> JudgeResult – same as Intermediate

calibration_set = [
    (task_a, obviously_good_answer_1, expected="PASS"),
    (task_a, obviously_bad_answer_1, expected="FAIL"),
    (task_b, obviously_good_answer_2, expected="PASS"),
    (task_b, obviously_bad_answer_2, expected="FAIL"),
]

function test_judge_calibration():
    for task, answer, expected in calibration_set:
        result = score_output(task, answer, rules)
        assert result.verdict == expected, f"judge got a known-{expected} case wr

function test_judge_is_self_consistent():
    verdicts = [score_output(task, answer, rules).verdict for _ in range(3)]
    assert len(set(verdicts)) == 1, f"judge disagreed with itself: {verdicts}"

```

Here's almost the whole thing — fill in one obviously-bad calibration case yourself:

```

CALIBRATION_SET = [
    {
        "task": "Summarize this ticket about a late delivery.",
        "answer": "The customer's order shipped late due to a warehouse delay; th
        "expected_verdict": "PASS",
    },
    {
        "task": "Summarize this ticket about a late delivery.",
        "answer": "I like pizza.", # obviously irrelevant – your turn: add one m
        "expected_verdict": "FAIL",
    },
]

def test_judge_calibration() -> None:
    for case in CALIBRATION_SET:
        result = score_output(case["task"], case["answer"], rules=["mentions the

```



```
assert result.verdict == case["expected_verdict"], (  
    f"judge got a known-{case['expected_verdict']} case wrong: {result.re  
)
```

Add one more obviously-bad case (too long, confidently states a fact not in the ticket) and a self-consistency test, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build the judge itself — the prompt, the call, the parsing. Advanced doesn't add anything to the judge's own logic; it adds a *test of the judge*, using a small hand-labeled calibration set and a self-consistency check, exactly the kind of "test the tests" thinking this document's Failure exercise applies to the whole suite — here applied one level lower, to the judge alone, before you ever trust it to gate a real regression check.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)